Enforcing a convention — a takeaway card

Lab 2.b built one guardrail in this training repo. This card is for doing it in **your own** codebase, where the conventions are yours and the stakes are real. It uses stored procedures as the worked example because that's a rule many teams have written down and almost nobody checks — but the shape applies to any convention.

Nothing here needs to be done in a lab. It's the independent follow-up; office hours will help.

The shape: one rule, three layers

(Layer numbers refer to Lab 2.b's five-layer model: 1 instruction · 2 Claude hooks + permissions · 3 commit-time · 4 CI · 5 bypass. This card uses 1, 2, and 4 — commit-time hooks are skipped, and the bypass appears inside layer 2's check.)

Take a rule you already have. *"Stored procedures use parameters — never build executable SQL by concatenating strings."* Most teams have said this out loud. Very few can tell you when it was last violated.

Layer 1 — instruction

One line where the agent always sees it:

```
- Don't: build executable SQL by string concatenation in a stored procedure —
        parameters only.
  Dynamic SQL requires `-- APPROVED-DYNAMIC-SQL: <reason + reviewer>`.
```

This informs. It does not enforce. Someone will ask for a flexible search and the agent will concatenate, politely and with good intentions.

Layer 2 — deterministic, while the code is being written

A `PreToolUse` hook that inspects the **content** of what's being written, not just the path. That's the difference from a rule like "don't touch this file":

```
#!/usr/bin/env node
let raw = '';
process.stdin.on('data', (c) => (raw += c));
process.stdin.on('end', () => {
  let filePath = '',
    body = '';
  try {
    const p = JSON.parse(raw);
    filePath = p.tool_input?.file_path ?? '';
    body = p.tool_input?.content ?? p.tool_input?.new_string ?? '';
  } catch {
    process.exit(0);
  }

  if (!/\.sql$/i.test(filePath)) process.exit(0);
  if (/--\s*APPROVED-DYNAMIC-SQL:/i.test(body)) process.exit(0); // documented bypass

  const declaresBuffer = /DECLARE\s+@\w+\s+n?varchar\s*\(\s*max\s*\)/i.test(body);
  const concatenatesParam = /\+\s*@/.test(body);

  if (declaresBuffer && concatenatesParam) {
    console.error(
      [
        'BLOCKED: this procedure builds executable SQL by string concatenation.',
        'Why: concatenating a parameter into a statement is an injection path.',
        'Instead: pass values as parameters, or use a static statement.',
        'Done means: no parameter is concatenated into SQL text, or the file
        carries',
        '  `-- APPROVED-DYNAMIC-SQL: <reason + reviewer>`.',
      ].join('\n'),
    );
    process.exit(2);
  }
  process.exit(0);
});
```

Layer 4 — the same rule at merge time

The hook stops the agent. This stops anyone:

```
# in YOUR repo's CI — lint-sql.mjs is a script you write (~50 lines; it doesn't exist
      here)
- name: SQL conventions
  run: node scripts/lint-sql.mjs # same rule module the hook imports
```

One rule module, two enforcement points. Write the check once and call it from both, or the two will drift and you'll trust the wrong one.

Be honest about the false positives

The heuristic above has real ones. `sp_executesql` with a *static* statement and bound parameters is correct practice. A procedure that declares an `nvarchar(max)` for an unrelated reason near a `+ @` trips it.

This isn't a flaw to fix before shipping — it's the decision you have to make:

- **narrow the trigger** and accept that some violations slip through, or
- **keep it broad** and route the noise to a nudge rather than a block, or
- **move the check to review** and stop pretending it's automated.

All three are legitimate. Silently shipping a broad blocking rule is not: the first time it fires on real work, someone disables hooks, and you've lost the layer entirely.

Which clauses can a gate actually decide?

Write your rule out and sort it before you write any code. This is the whole exercise — and this table **is** the marking convention Lab 2.a's NFR asks for, with its three verdicts: mechanical, partly (heuristic), and human-review.

Candidate clause	Mechanical?
Header block present (Purpose / Params / Returns / Errors)	Yes — structural
No `SELECT *` in a returned result set	Yes
`SET NOCOUNT ON` as the first statement	Yes
Writes wrapped in an explicit transaction with `XACT_ABORT ON`	Yes
No `DROP` / `TRUNCATE`; no `DELETE` / `UPDATE` without a `WHERE`	Yes
Parameters only — no concatenation into executable SQL	**Partly** — heuristic, expect false positives
"Purpose describes what, not how"	No → human review
Counts computed before paging	No, semantic → human review

The bottom three are the point. **Every real convention has a residue that tooling cannot hold**, and each residual clause needs a recorded decision: accepted risk, human gate, or dropped — with a name

against it. A standard that pretends to be fully enforced is worse than one that is honest about its edges, because people trust it further than it deserves.

A template worth stealing

If you do write the convention down, give each procedure a header contract. It costs a few lines and makes the structural clauses above checkable:

```
-- ============================================================
-- Procedure: usp_<Entity>_<Verb>
-- Purpose:   <one line — what it does, not how>
-- Mode:      READ | WRITE
-- Params:    @<Name> <type>   -- <meaning>
-- Returns:   <result shape, or ROWCOUNT>
-- Errors:    <named condition → what the caller sees>
-- NFR:       <link to the rule this obeys>
-- ============================================================
```

A second worked example — decision → rule → gate, org-wide

The stored-procedure convention above is one repo's rule. Here is the same shape on a rule whose reach is **every** repo — worth walking because the word "all" changes where everything lives.

Suppose the team decides: *"all backends that use a database must use SQLite."*

Document	What it holds	Why there and not elsewhere
ADR — "SQLite for backend persistence"	The *decision*: context (ops burden, deployment simplicity), options considered (Postgres, MySQL), and an honest cost (concurrent-write limits; a service that outgrows it needs a superseding ADR)	ADRs are history — superseded, never edited. **The word "must" does not belong here**: a rule buried in an accepted ADR is invisible at the moment of violation
NFR — "Database engine"	The *statute*: "Backend services persist via SQLite only. **Checked by:** CI dependency scan — any of `pg`, `mysql2`, `mongodb` in a server manifest fails the gate. Rationale: ADR-00XX"	NFRs are law — revised in place, and obliged to say how compliance is checked
Guardrail	A deny/ask entry on `npm install pg*` (authoring time) + the CI dependency scan (merge time)	Same rule, two enforcement points — the agent and everyone else

Two things this example shows that a single-repo rule can't:

- **“All backends” makes the rule Collective-layer.** Its reach exceeds any one repo’s `docs/` folder, so the ADR + NFR belong in the org’s governance repo, with each backend repo carrying a pointer (and inheriting the gate). That is exactly the problem `distributing-standards.md` exists to solve.
- **The pair is real but asymmetric.** The ADR without the NFR is a decision nobody can act on at review time; the NFR without the ADR is a rule that reads as arbitrary the first time someone wants Postgres. They cross-reference — “rationale: ADR-00XX” / “enforced via NFR-00XX” — and neither restates the other.

Start here

1. Pick **one** rule you already have written down and cannot currently check.
2. Sort its clauses into mechanical and fuzzy.
3. Build the mechanical core as a nudge first. Watch what it catches for a week.
4. Promote it to a block only once you’ve seen it not fire on legitimate work.
5. Write down the residue, the bypass, and who owns each.

Step 5 is the one people skip, and it’s the one that makes the rest trustworthy.